GIT CHEAT SHEET

Terminal only (prompt $ or %), never the R Console (>). Habit: branch, status, add, commit, push

1. LOOK FIRST (SAFE)

`git status`	branch + staged/unstaged/untracked
`git branch`	list branches, * = where you are
`git log --oneline -5`	last 5 commits
`git diff`	unstaged line changes
`git diff --staged`	what is about to be committed
`git remote -v`	https:// = helper, git@ = SSH

Fun fact: Linus Torvalds wrote git in about 10 days in 2005.

2. SAVE LOOP

`git add <file>`	put it in the box (stage)
`git commit -m "msg"`	seal and label it (local snapshot)
`git push`	ship the box to GitHub
`git push -u origin <branch>`	first push of a new branch

Staging is not saving. add without commit = nothing to push. Stage by name, avoid add -A.

3. BRANCHES

`git checkout -b feature-ndvi`	create and switch
`git switch -c feature-ndvi`	newer form, same thing
`git checkout main`	switch to an existing branch
`git branch -d <name>`	delete merged branch (safe)
`git branch -D <name>`	force delete (careful)

A branch is a tiny file holding one commit ID, so make them freely. Uncommitted edits follow you to the new branch.

4. MERGING

`git checkout main`	go to the RECEIVING branch
`git merge feature-ndvi`	bring it into main
`git merge --abort`	cancel a scary merge
`git add <file> then git commit`	finish a conflict fix

Fast-forward = silent. Merge commit = vim opens. Conflict = delete the <<<<<<< ======= >>>>>>> markers.

5. GITHUB SYNC

`git clone <url>`	download a repo (once)
`git fetch origin`	peek at GitHub, no changes
`git pull`	fetch + merge
`git pull --no-rebase origin main`	pull and merge both sides
`git remote set-url origin <url>`	repoint origin
`git remote add origin <url>`	first setup only

origin is just a nickname. "fetch first" = GitHub has commits you lack: pull, then push.

6. UNDO AND RESCUE

`git restore <file>`	PERMANENT: discard edits
`git restore --staged <file>`	unstage, keep edits
`git reset`	unstage all, files untouched
`git checkout -f <branch>`	DESTRUCTIVE: back up first
`git stash and git stash pop`	shelve and restore edits
`git reflog`	history of HEAD, the rescue tool
`git rm <file>`	delete and stage the deletion

Committed work is very hard to lose (reflog keeps about 90 days). Uncommitted work is not.

7. NEW REPO: CLONE OR INIT?

`README ticked on GitHub`	git clone <url>, then cd in
`Repo created empty`	git init -b main
`then`	git remote add origin <url>
`then`	git push -u origin main
`Already cloned`	cd in. Do not clone again

Wrong choice = "rejected (fetch first)" on the first push. Run pwd before init so you never nest a repo inside a repo.

8. FORK AND PULL REQUEST

`1. Fork`	button on the repo page
`2. git clone <your-fork-url>`	copy your fork
`3. git checkout -b add-topic`	new branch
`4. git add, git commit`	save your note
`5. git push -u origin add-topic`	send the branch
`6. Compare & pull request`	on GitHub, submit

Strangers cannot push to a public repo, so they fork and ask the owner to pull their branch in.

9. SHELL BASICS

`pwd`	which folder am I in?
`ls and ls -la`	list files, and hidden ones
`cd <folder> and cd ..`	move in, move up
`cp <file> .`	copy here
`echo "x" > file`	write a file (REPLACES it)
`q`	leave git diff or git log output
`Ctrl+C`	cancel a stuck prompt

10. SETUP (ONE-TIME)

`git config --global user.name "Name"`	name on commits
`git config --global user.email "e@x.com"`	email on commits
`git config --global --get-all credential.helper`	want only one helper
`git credential-manager configure`	register GCM
`git config --global core.editor nano`	swap out vim

11. VIM ESCAPE

`Esc :wq Return`	save and finish
`Esc :q! Return`	quit, discard

Exiting vim is a top Stack Overflow question. You are in good company.

12. ERROR DECODER

`fetch first`	GitHub ahead: git pull, then push
`non-fast-forward`	git pull --no-rebase origin main
`remote origin already exists`	use set-url, not add
`Already up to date (merge)`	wrong branch: checkout main first
`not a git repository`	wrong folder: pwd, cd
`CONFLICT in file`	fix markers, add, commit
`exceeds 100 MB limit`	gitignore it, git rm --cached
`stuck > or function>`	wrong window: Ctrl+C
`unexpected symbol`	git typed in the R Console

13. EARTH OBSERVATION HYGIENE

`Rasters (.tif .nc)`	do not commit, link the source
`GitHub file limit`	100 MB per file
`Tokens and keys`	never commit, revoke if leaked
`.gitignore`	write it before commit 1
`Reproducible`	script plus data recipe, not data

Deleting a leaked token's file does not remove it from history. Revoke the token.

14. DAILY WORKFLOW

`1. git branch, git status`	where am I? clean?
`2. edit files`	
`3. git diff`	what changed?
`4. git add <file>, git status`	right files staged?
`5. git commit -m "..."`	save
`6. git push`	send